

A Reproducible Pipeline for Shortcut Learning Evaluation and Mitigation in Large Language Models

Shortcut Learning Project Team

March 6, 2026

Abstract

We present a fully implemented, reproducible pipeline for diagnosing and mitigating shortcut learning in language models. The system covers dataset preprocessing, training under multiple optimization regimes, shortcut stress testing, robust evaluation metrics, mitigation modules, and experiment reporting. In addition to standard accuracy and worst-group accuracy, we implement semantic and explanation-aware indicators including Semantic Fidelity Score (SFS), Internal Consistency Score (ICS), Explanation Quality Score (EQS), and a confidence proxy (CFS). The pipeline is modular and designed for extension to new tasks and shortcuts.

1 Introduction

Shortcut learning is the tendency of models to rely on spurious correlations instead of robust causal signals. This can inflate in-distribution scores while causing brittle out-of-distribution behavior. Our objective is to provide a practical end-to-end framework for:

1. generating and loading benchmark and shortcut-augmented datasets,
2. training models under controlled optimizer/batch-size settings,
3. quantifying shortcut sensitivity using standard and explanation-aware metrics,
4. applying mitigation methods and comparing outcomes.

2 Pipeline Design

The implemented repository follows a staged architecture:

1. **Data preprocessing:** synthetic HANS/NLI-like, SST-2 trigger, and MCQA data generators with augmentation hooks.
2. **Data loading:** PyTorch `Dataset` classes with metadata-aware collators.
3. **Training:** HuggingFace model loading, Adam/SGD optimizers, optional gradient accumulation, and small-batch β_2 scaling.
4. **Evaluation:** accuracy, worst-group accuracy, SFS, ICS, EQS, CFS.
5. **Stress tests:** MCQA option permutation and SST-2 trigger mutation checks.
6. **Mitigation:** counterfactual text augmentation and GroupDRO-style group-robust objective.

3 Implemented Metrics

3.1 Accuracy and Worst-Group Accuracy

Let $\hat{y}_i$ and y_i be prediction and label. Accuracy is

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i].$$

For groups $g \in \mathcal{G}$, group accuracy is computed per group and worst-group accuracy is

$$\text{WGA} = \min_{g \in \mathcal{G}} \text{Acc}_g.$$

3.2 Semantic Fidelity Score (SFS)

We compute cosine similarity between normalized bag-of-words embeddings of prompt and model output as a lightweight semantic proxy:

$$\text{SFS}(p, o) = \cos(\phi(p), \phi(o)).$$

3.3 Internal Consistency Score (ICS)

ICS is defined as a contradiction-sensitive binary score over explanation steps. The provided implementation flags direct negation conflicts and returns 0 upon contradiction, else 1.

3.4 Explanation Quality Score (EQS)

$$\text{EQS} = 0.5 \cdot \text{SFS} + 0.5 \cdot \text{ICS}.$$

3.5 Confidence Score (CFS)

CFS is parsed from model confidence-formatted output and normalized to $[0, 1]$.

4 Training with Optimizer Scaling

For Adam under small batches, we scale β_2 by token half-life:

$$\beta_2 = \exp\left(\frac{\log(0.5)}{H/B}\right),$$

where H is desired half-life in tokens and B is batch size.

5 Mitigation Modules

5.1 Counterfactual Augmentation

We provide text replacement mapping to create counterfactual samples and re-balance spurious features.

5.2 GroupDRO-style Objective

Given per-example losses grouped by metadata, we compute group mean losses and a softmax-reweighted robust loss.

6 Experiment Protocol

Recommended ablation grid:

- models: DistilBERT, GPT-2 family, LLaMA-family
- batch size: {1, 8, 32, 128}
- optimizers: Adam (scaled β_2), SGD
- prompts: zero-shot, few-shot, CoT
- datasets: NLI/HANS, SST-2 trigger, MCQA permutation

7 CLI and Reproducibility

All core stages are executable via Python CLI scripts: preprocessing, training, evaluation, and shortcut tests. Reproducibility support includes seeded randomness, deterministic PyTorch settings, and environment pinning (`requirements.txt`, `environment.yml`, `Dockerfile`).

8 Limitations

The default repository uses lightweight synthetic preprocessing for immediate reproducibility and CI friendliness; users should replace with full benchmark loaders for production experiments. SFS/ICS are intentionally lightweight defaults and can be swapped with embedding/NLI model-based implementations.

9 Conclusion

This work delivers an implementation-ready baseline for shortcut-learning research in LLMs. The modular structure simplifies adding new datasets, metrics, and mitigation strategies while preserving reproducibility and experiment traceability.

A Commands

```
python scripts/preprocess.py --dataset hans --augment negation --output_dir data/hans
python scripts/train.py --model distilbert-base-uncased --dataset_dir data/hans --output_dir m
python scripts/evaluate.py --model_dir models/hans --dataset_path data/hans/val.jsonl
python scripts/test_shortcuts.py --dataset_path data/mcqa/val.jsonl --tests permute_mcqa
```